LAB REPORT – LLMNR Poisonig

Summary

In this report, I will demonstrate a vulnerability found in the Windows operating system when using the LLMNR protocol. This weakness allows an attacker to impersonate a trusted server and trick a user into sending NTLM-based authentication credentials.

The demonstration includes a practical example using Wireshark, along with an explanation of theissue and its mitigation.

Phase 1: Preparation for an LLMNR Poisoning Attack

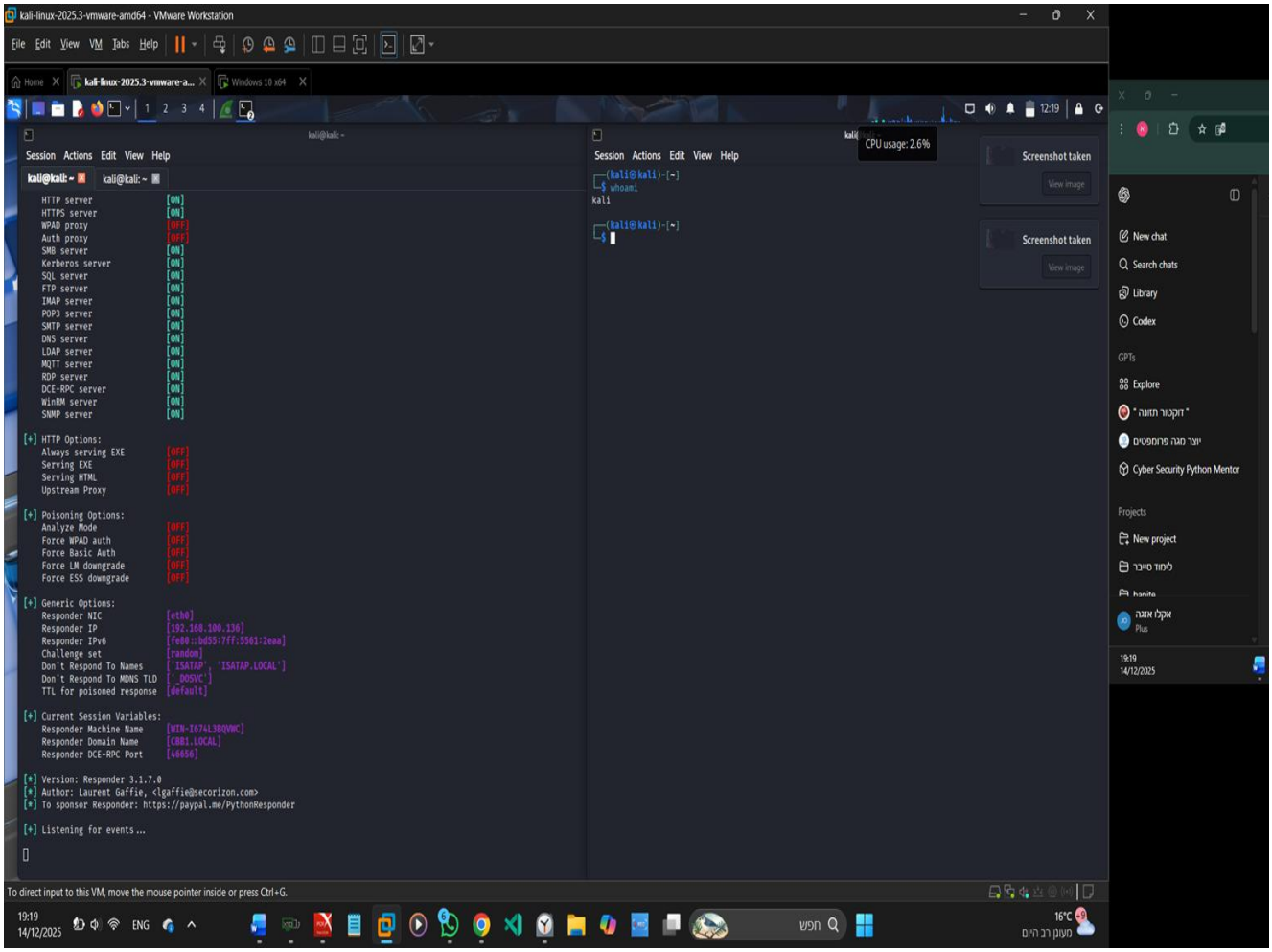
The attacker, named Almog, runs a poisoning tool on his Kali Linux machine called **Responder**.

The command used is:

```
sudo responder -I eth0
```

This tool allows the attacker to listen and respond to network traffic. Since he is connected to the same network, he waits for a situation where a user attempts to access a server but either mistypes its name or the server cannot be found.

At that point, the attacker responds to the request and impersonates the legitimate server.

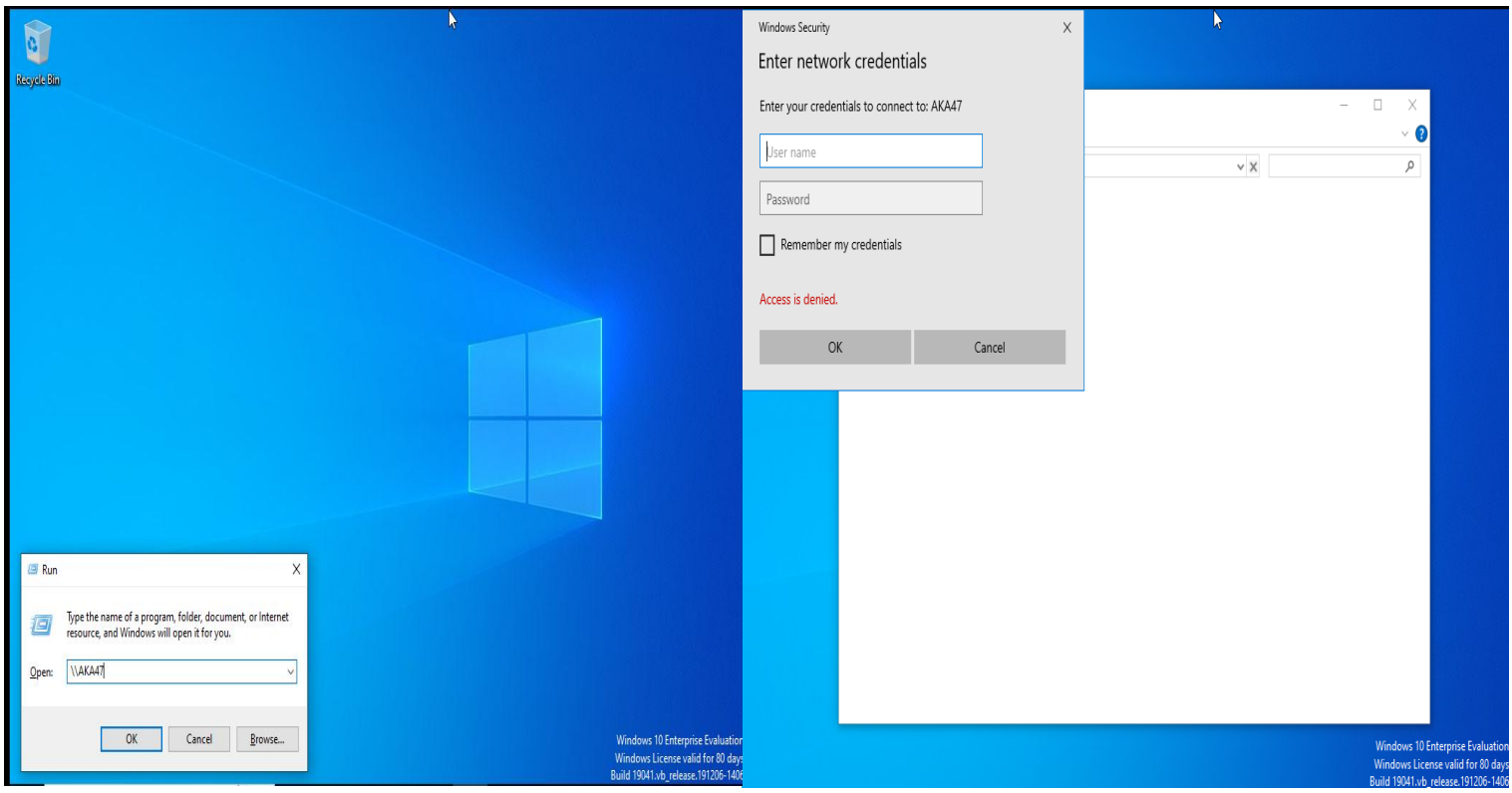


hase 2: DNS Resolution Failure and Use of LLMNR

A user named Mordechai attempts to access a server called `AKA47\\`, but the system fails to resolve the hostname using DNS.

As a result, Windows automatically falls back to the **LLMNR (Link-Local Multicast Name Resolution)** protocol to try and locate the requested resource on the local network.

This behavior creates an opportunity for an attacker to intercept the request and respond before a legitimate server does..



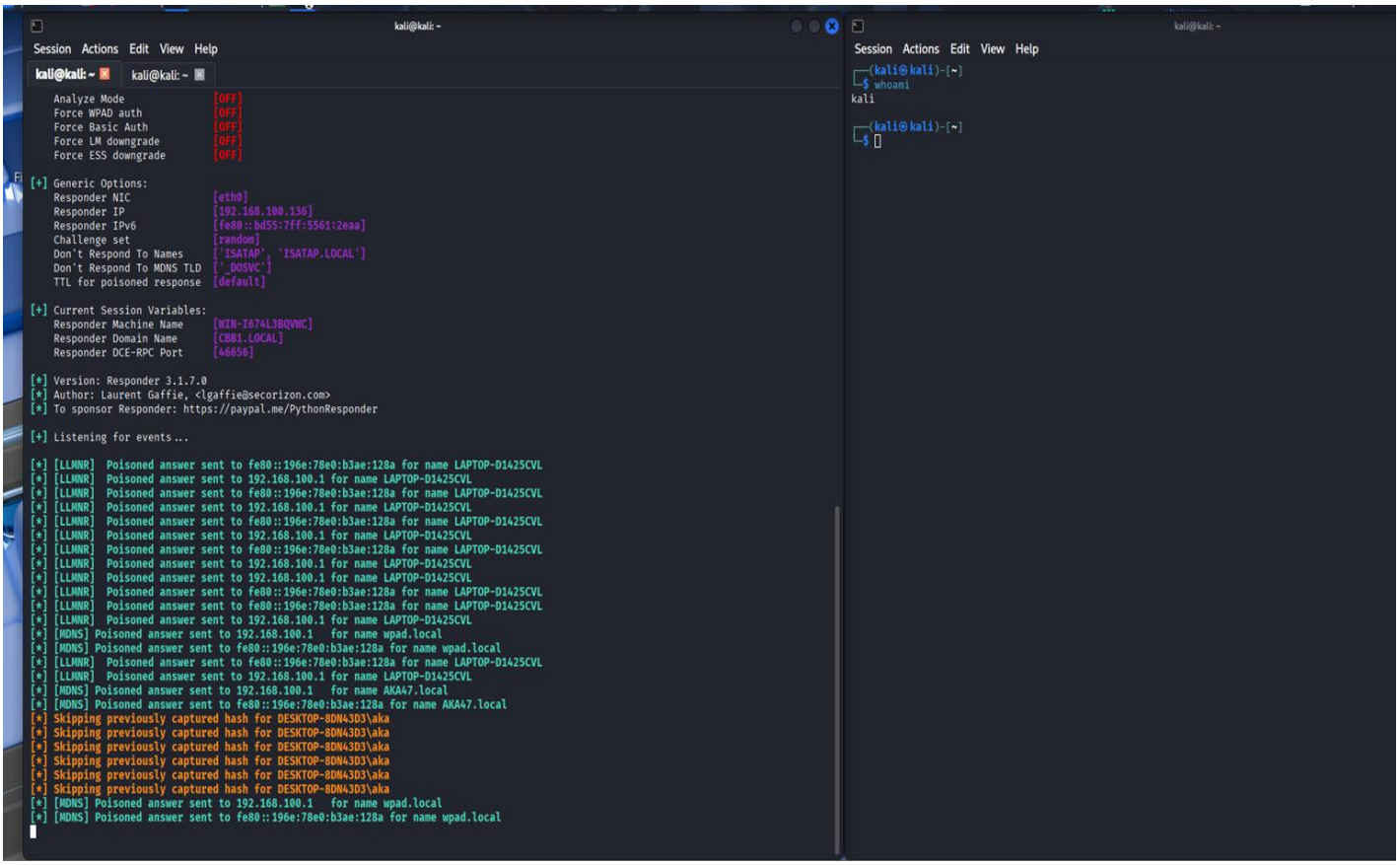
Phase 3: The Attacker Captures the NTLM Hash

Since the DNS server fails to resolve the requested hostname, the system falls back to using the **LLMNR** protocol.

LLMNR sends multicast requests across the local network, asking if any device recognizes the requested hostname.

The attacker (Almog) responds to this request, claiming to be the legitimate server. Because LLMNR does not provide proper authentication, the victim's machine trusts the response and initiates an authentication process with the attacker's machine.

As part of this process, Mordechai's system sends NTLM authentication data, including the hashed credentials (NTLM hash), to the attacker.

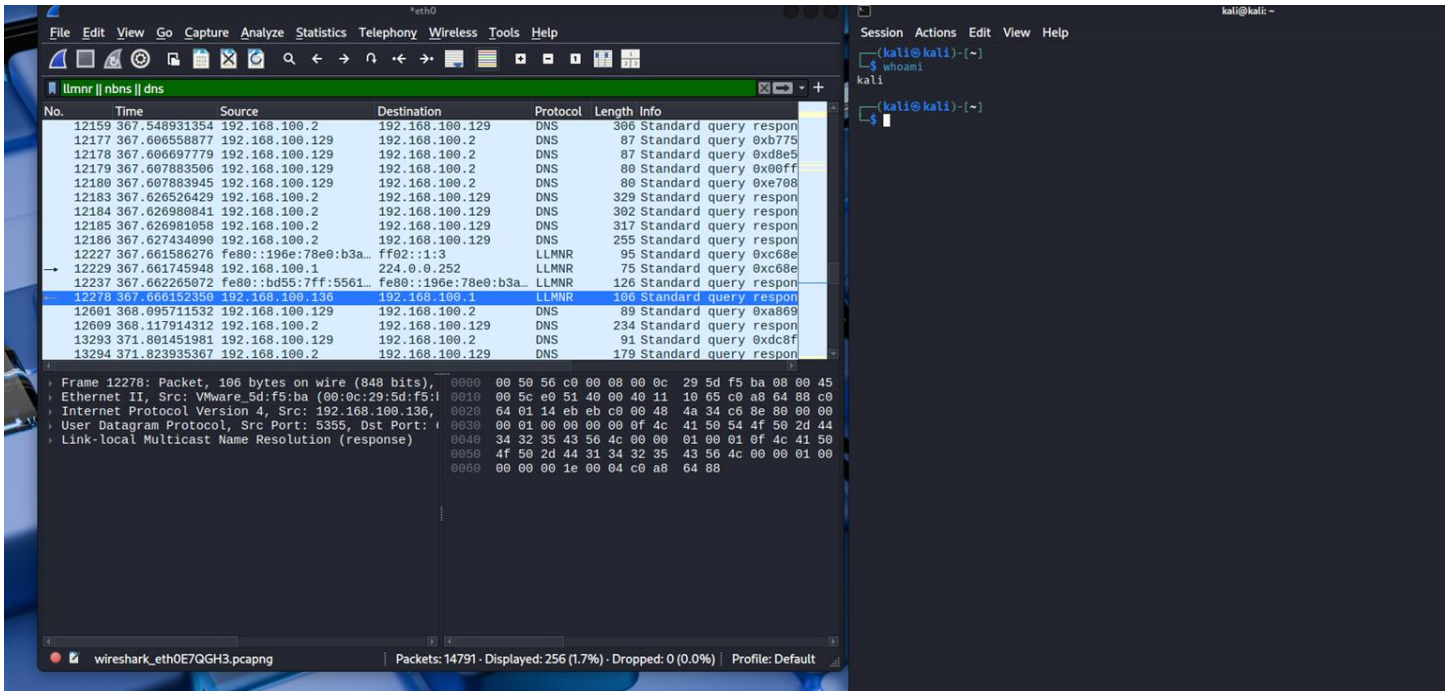


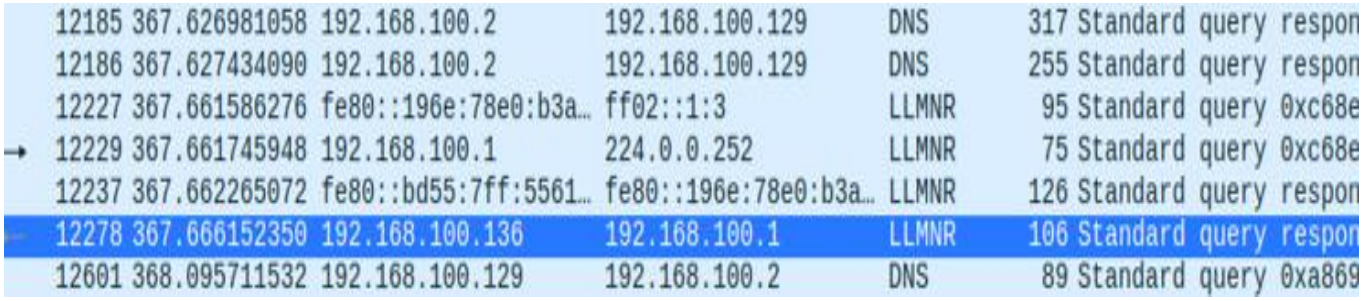
Phase 4: Evidence in Wireshark

To validate the attack, network traffic was captured and analyzed using Wireshark.

During the analysis, LLMNR multicast requests can be observed, showing the victim attempting to resolve the hostname. Shortly after, a response from the attacker's machine appears, indicating a spoofed reply.

In addition, NTLM authentication traffic is visible, confirming that the victim's system initiated an authentication process with the attacker. This includes the transmission of NTLM challenge-response data, which can be captured and used for further attacks (e.g., relay or cracking)





Additional Observation (Wireshark Analysis)

It can be observed that when the DNS protocol fails to resolve the requested hostname, the system falls back to using the **LLMNR** protocol.

LLMNR sends multicast requests twice:

- Once over IPv4 to the multicast address **224.0.0.252**

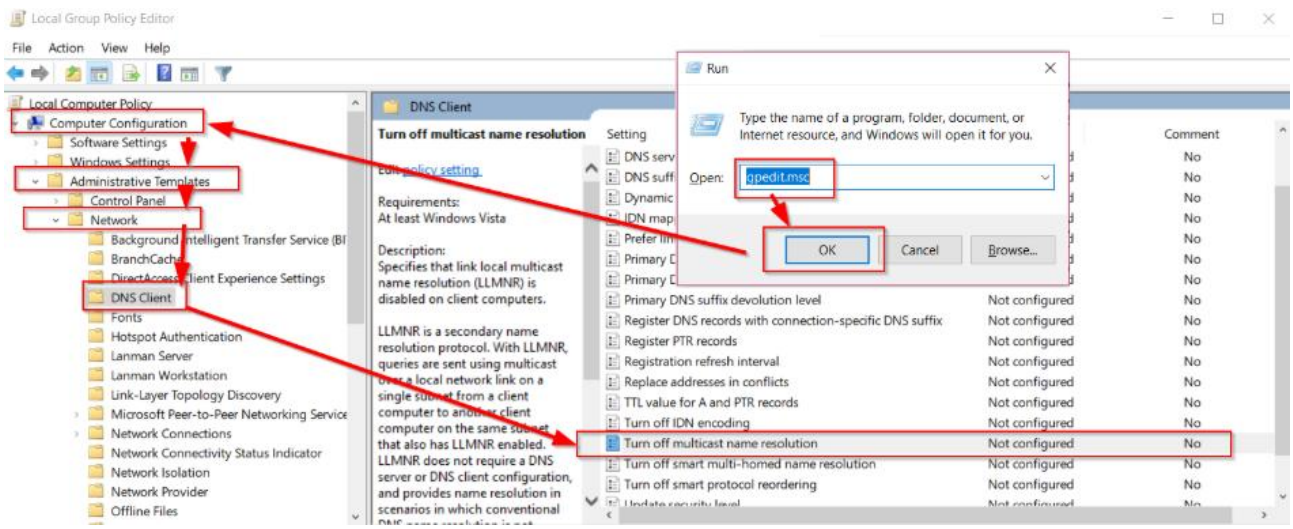
- Once over IPv6 (to the corresponding multicast address)

Following these requests, the attacker's machine (Almog) responds, claiming ownership of the requested hostname. The response includes the attacker's IP address: **192.168.100.136**.

This confirms that the attacker successfully spoofed the response and positioned himself as the target server.

Phase 5: Mitigation

The most effective way to mitigate this vulnerability, until a more robust solution is implemented, is to disable the **LLMNR protocol** using **Group Policy (GPO)**.

This can be configured in the Group Policy settings, as shown in the image above, by disabling multicast name resolution.

By doing so, the system will no longer fall back to LLMNR when DNS resolution fails, preventing attackers from exploiting this behavior to capture NTLM credentials.